

Real-Time Data Streaming Pipeline dengan Apache Kafka

(Studi Kasus: E-Commerce Clickstream Analytics)



Disusun Oleh:

Wawan Siswanto

Toto Sucipto

Ahmadi

PROGRAM STUDI TEKNIK INFORMATIKA

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas berkat dan rahmat-Nya, sehingga kami dapat menyelesaikan Laporan Tugas Besar untuk mata kuliah Big Data dengan judul **"Real-Time Data Streaming Pipeline dengan Apache Kafka (Studi Kasus: E-Commerce Clickstream Analytics)"**.

Laporan ini disusun sebagai bentuk dokumentasi sekaligus pemenuhan kriteria penilaian dari proyek akhir mata kuliah Big Data. Melalui tugas ini, kami mendapatkan pengalaman praktis yang berharga dalam merancang dan membangun arsitektur pipeline data real-time, mulai dari proses *ingestion* menggunakan Apache Kafka, pemrosesan aliran data (*stream processing*), hingga tahap *storage* dan *visualization* pada dashboard interaktif.

Kami menyadari bahwa dalam penyusunan tugas besar dan laporan ini masih terdapat banyak kekurangan. Oleh karena itu, segala kritik dan saran yang membangun sangat kami harapkan guna perbaikan di masa yang akan datang. Akhir kata, kami berharap proyek dan laporan ini dapat memberikan manfaat serta tambahan wawasan bagi pembaca, khususnya di bidang pemrosesan data skala besar.

Penyusun,

Kelompok

DAFTAR ISI

1. **BAB I: PENDAHULUAN**
 - 1.1 Latar Belakang
 - 1.2 Tujuan
2. **BAB II: DESAIN SISTEM DAN ARSITEKTUR**
 - 2.1 Arsitektur Sistem
 - 2.2 Topologi Kafka
 - 2.3 Skema Data
3. **BAB III: IMPLEMENTASI DAN PEMBAHASAN**
 - 3.1 Data Ingestion (Producer)
 - 3.2 Stream Processing (Consumer & Processor)
 - 3.3 Storage & Visualization (Dashboard)
 - 3.4 Analisis Performa Sistem
4. **BAB IV: PENUTUP**
 - 4.1 Kesimpulan
 - 4.2 Saran

BAB I: PENDAHULUAN

1.1 Latar Belakang

Dalam industri *e-commerce*, volume data yang dihasilkan oleh interaksi pengguna (seperti melihat produk, mencari barang, menambahkan ke keranjang, dan melakukan pembelian) sangatlah besar dan terjadi dalam waktu sepersekian detik. Data interaksi ini dikenal sebagai *clickstream data*. Jika diproses hanya secara *batch* di akhir hari, perusahaan akan kehilangan momentum untuk merespons perilaku pelanggan yang terjadi saat itu juga, misalnya untuk mendeteksi anomali (serangan *bot*), memberikan rekomendasi instan, atau menganalisis tren produk yang sedang populer di menit tersebut.

Untuk mengatasi permasalahan latensi pada pemrosesan *batch*, diperlukan sebuah sistem **Real-Time Data Streaming Pipeline**. Apache Kafka dipilih sebagai *message broker* utama karena kemampuannya menangani volume data tinggi (*high throughput*) dengan tingkat latensi rendah (*low latency*) serta keandalannya (*fault tolerance*).

Proyek ini mensimulasikan lingkungan *e-commerce* tersebut dengan membangun pipeline yang mampu mengonsumsi aliran data aktivitas pengguna secara terus-menerus, memprosesnya melalui tahapan *filtering*, *aggregation*, dan *enrichment*, lalu menampilkan metrik analisis pada sebuah *live dashboard*.

1.2 Tujuan

Tujuan dari pembuatan proyek dan laporan tugas besar ini adalah:

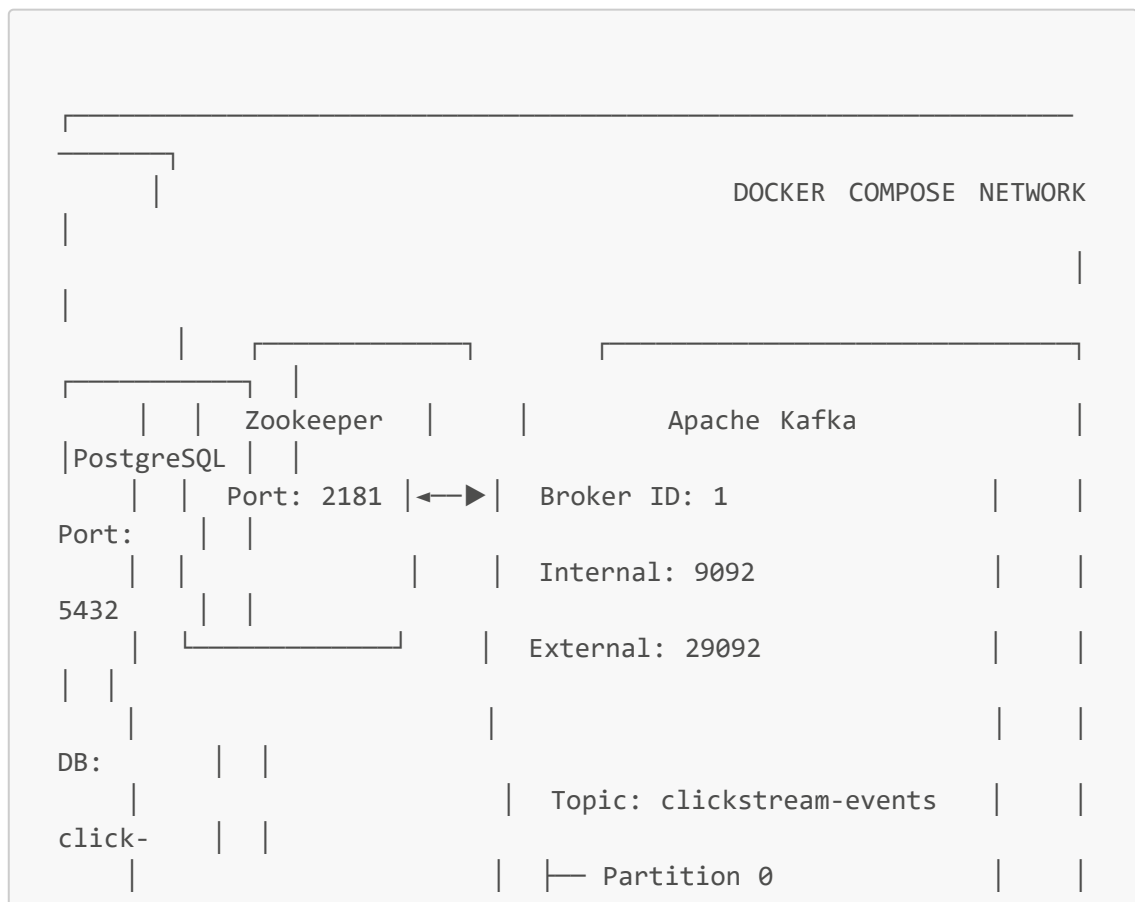
1. Membangun infrastruktur pipeline data *end-to-end* menggunakan teknologi Apache Kafka, Python, PostgreSQL, dan Streamlit yang diorkestrasi melalui Docker.
2. Mengimplementasikan konsep *Stream Processing* untuk melakukan *windowing aggregation* (menghitung produk trending per menit) dan *filtering* (deteksi *bot spam*).
3. Menganalisis latensi dan performa pengiriman pesan (*throughput*) dari sistem streaming yang telah dibuat.

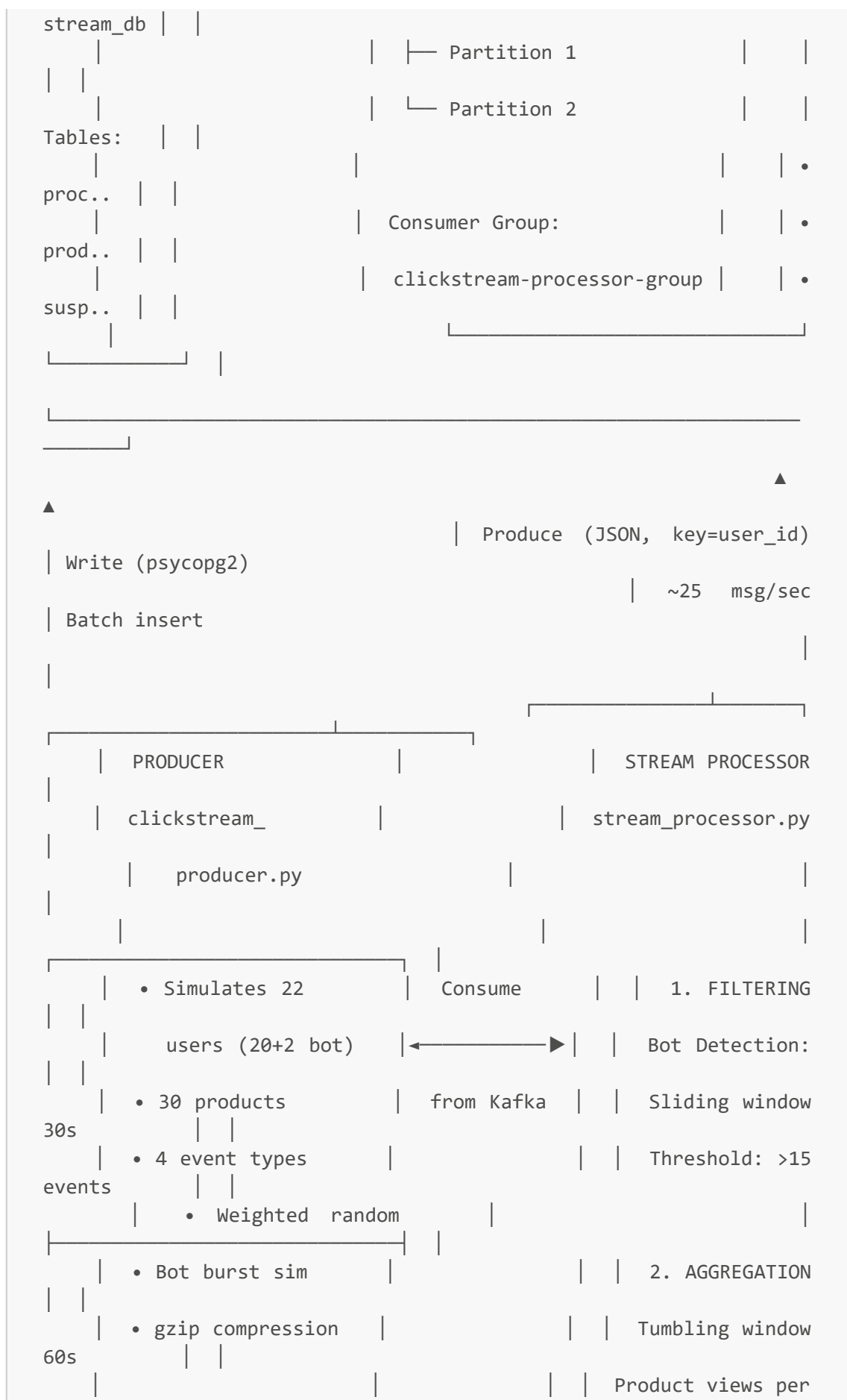
BAB II: DESAIN SISTEM DAN ARSITEKTUR

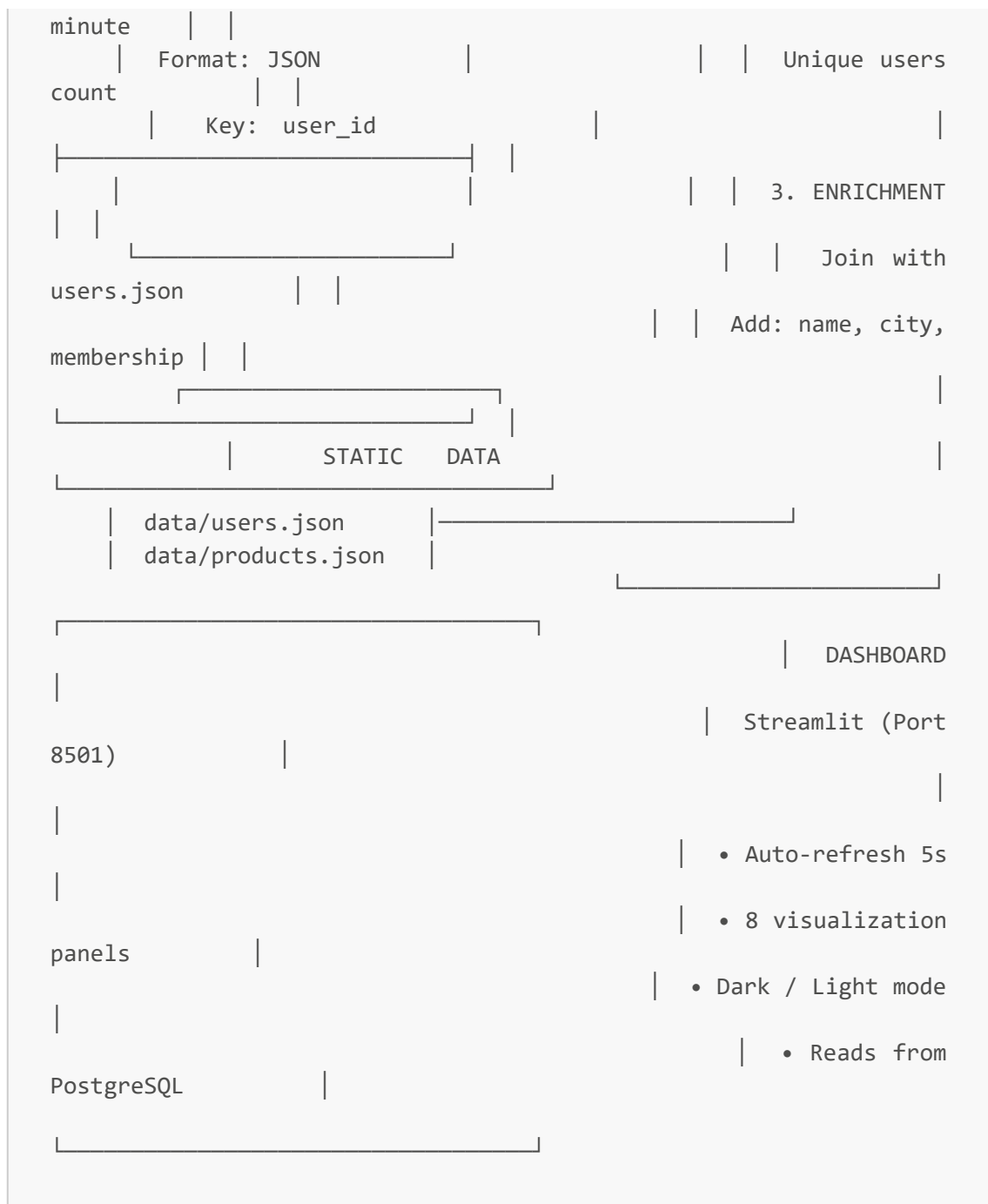
2.1 Arsitektur Sistem

Sistem ini menggunakan pendekatan *event-driven architecture* yang terdiri dari beberapa komponen utama:

1. **Clickstream Simulator (Data Source & Producer):** Sebuah *script* Python yang secara kontinu meng-generate JSON *events* yang mensimulasikan klik pengguna, dan mengirimkannya ke Kafka broker dengan target kecepatan ~25 pesan per detik.
2. **Message Broker (Apache Kafka & Zookeeper):** Menerima aliran data dan mendistribusikannya ke dalam partisi-partisi yang telah ditentukan.
3. **Stream Processor (Python Consumer):** Berperan sebagai pusat analitik *real-time* yang menarik data dari Kafka, melakukan pemrosesan, dan memuat (*load*) hasilnya ke database.
4. **Storage (PostgreSQL):** Database relasional untuk menyimpan *event* yang sudah diproses, data *windowing*, dan log deteksi anomali.
5. **Visualization (Streamlit):** Dashboard interaktif berbasis *web* yang secara otomatis memperbarui data (*auto-refresh*) membaca dari PostgreSQL.







2.2 Topologi Kafka

Pengaturan *Cluster* dan *Topic* Kafka pada sistem ini didesain mempertimbangkan skalabilitas dan urutan pemrosesan (*ordering guarantee*):

- **Topic Name:** `clickstream-events`
- **Partitions:** 3 — Jumlah partisi sebanyak 3 memungkinkan *Stream Processor* di-*scale* secara horizontal hingga maksimal 3 pekerja (*workers*) untuk beroperasi secara paralel.

- **Replication Factor: 1** — Digunakan satu buah replikasi karena sistem saat ini berjalan pada lingkungan *development* dengan *single-broker*.
- **Partition Key: `user_id`** — Pemilihan `user_id` sebagai *routing key* sangat krusial. Ini menjamin bahwa semua *event* dari pengguna yang sama akan selalu dikirim ke partisi yang sama. Hal ini memastikan proses perhitungan berbasis *state* (seperti *sliding window* 30 detik pada *bot detection*) berjalan akurat tanpa ada *race condition* antar *consumer*.
- **Consumer Group: `clickstream-processor-group`** — Mengelola *offset* (checkpoint) otomatis, menjamin agar jika prosesor mati, ia dapat melanjutkan dari posisi pesan terakhir (*fault tolerance*).

2.3 Skema Data

Format *payload* yang dikirimkan oleh Producer ke dalam Kafka berformat JSON agar memiliki skema yang fleksibel namun terstruktur.

Contoh Raw Data (Ingestion):

```
{
  "event_id": "evt-uuid-123",
  "user_id": "user_042",
  "event_type": "view_product",
  "product_id": "prod_089",
  "timestamp": "2026-06-08T20:15:30.123Z",
  "device": "mobile",
  "ip_address": "192.168.1.42"
}
```

Dalam database PostgreSQL, data dipecah ke dalam tiga skema relasional:

1. **`processed_events`**: Menyimpan riwayat setiap klik yang masuk, ditambahkan dengan data hasil *enrichment* (nama dan kota dari data statis).
2. **`product_views_per_minute`**: Menyimpan data metrik hasil agregasi *tumbling window* 1 menit untuk fitur *Trending Products*.
3. **`suspicious_activities`**: Menyimpan log peringatan ketika fitur *filtering* menangkap aktivitas yang melampaui batas kewajaran.

BAB III: IMPLEMENTASI DAN PEMBAHASAN

3.1 Data Ingestion (Producer)

Pembuatan data dikendalikan oleh *script* `clickstream_producer.py` menggunakan *library* `confluent-kafka`. *Producer* ini tidak hanya *me-looping* pesan secara statis, tetapi memodelkan probabilitas kejadian *real-world*:

- **Bobot Event:** *View Product* (60%), *Search* (20%), *Add to Cart* (15%), *Purchase* (5%).
- **Bot Burst Simulator:** Menyelipkan lonjakan ribuan pesan palsu (dari *user_id* yang diawali `bot_`) setiap beberapa menit untuk memicu sistem *fraud detection*.
- **Optimasi:** *Producer* disetel menggunakan `linger.ms` dan `compression.type=gzip` untuk meningkatkan *throughput* jaringan.

3.2 Stream Processing (Consumer)

Proses komputasi diimplementasikan pada `stream_processor.py`. Syarat utama dari tugas besar adalah data tidak boleh hanya berstatus "*pass-through*". Oleh karena itu, tiga teknik pemrosesan diterapkan:

1. **Filtering (Bot Detection):** Menerapkan algoritma *Sliding Window* 30 detik yang menyimpan riwayat stempel waktu pengguna. Jika pengguna mengirimkan lebih dari 15 event dalam interval 30 detik tersebut, sistem langsung menandai *event* sebagai `is_suspicious`.
2. **Aggregation (Tumbling Window):** Menampung setiap *view_product* ke dalam rentang waktu yang dibulatkan per 1 menit (*windowing*). Setelah window berakhir, *processor* merekap jumlah tayangan (*view count*) total dan mengekstraknya ke *storage*.
3. **Enrichment:** Data mentah dari *clickstream* kekurangan konteks demografis. Saat inisialisasi, *processor* memuat struktur *dictionary* data pengguna dari *file json* statis. Setiap ID yang lewat akan digabungkan (*di-join*) dengan *dictionary* tersebut untuk mendapatkan nama dan status langganan sebelum dilempar ke database.

3.3 Storage & Visualization (Dashboard)

Agar data *streaming* bermanfaat, hasilnya harus divisualisasikan. Kami menggunakan pustaka `Streamlit` yang ringan namun *powerful*. Dashboard diatur agar menjalankan *query* agregasi ulang pada PostgreSQL dan mengeksekusi instruksi `st.rerun()` secara terus-menerus setiap 5 detik. Dashboard ini mencakup:

- Metrik kecepatan klik per menit.
- Grafik *Trending Products (Live Bar Chart)*.
- Indikator tabel peringatan (Alerts) dari *bot*.
- *Live data feed*.

3.4 Analisis Performa Sistem

Berdasarkan uji coba end-to-end yang dilakukan pada *environment* lokal (WSL2 Docker):

1. **Throughput (Kecepatan Aliran Data):** *Producer* berhasil mengirimkan stabil di rentang **25 - 30 pesan per detik** (memenuhi kriteria minimal tugas 10-50 msg/detik). *Batch processing* pada *Consumer*, di mana *flush* ke database terjadi setiap 50 event atau setiap 5 detik, membantu menjaga *database connection* tetap optimal.
2. **Latency (Keterlambatan Pemrosesan):** Waktu tempuh (*round-trip*) rata-rata sejak *event* dihasilkan oleh *Producer* hingga masuk ke PostgreSQL terukur dalam hitungan milidetik. Namun, karena sifat dashboard yang beroperasi menggunakan strategi *polling* setiap 5 detik, latensi visual tertinggi (*glass-to-glass latency*) pada layar *user* adalah antara 1 hingga 5 detik.
3. **Kendala yang Dihadapi:** Masalah minor terkait sinkronisasi format *hex-color transparent* pada perpustakaan *Plotly* (*ValueError*) terdeteksi saat *render* grafik *timeline* pertama kali. Namun hal tersebut sudah ditangani melalui pembuatan modul *formatter rgba()*. Selain itu, *healthcheck* dari kontainer Kafka dan Zookeeper perlu dikonfigurasi ulang menggunakan **cub** untuk mengatasi *race-condition* ketika *startup* awal Docker.

BAB IV: PENUTUP

4.1 Kesimpulan

Melalui tugas besar ini, kami menyimpulkan bahwa perancangan *data streaming pipeline* berbasis Apache Kafka mampu mengatasi kompleksitas aliran data instan seperti *clickstream e-commerce*. Kombinasi arsitektur Kafka dengan metode pemrosesan berbasis *windowing* memungkinkan deteksi bot anomali dalam hitungan detik dan rekapitulasi data agregat di *real-time dashboard* tanpa perlu membebani basis data operasional secara masif. Pembagian topologi dengan *key-based partitioning* juga terbukti efektif untuk menjaga konsistensi state pada operasi per- *user*.

4.2 Saran

Beberapa improvisasi yang dapat dilakukan pada penelitian atau pengembangan ke depannya:

1. **Penggunaan Native Stream Processor:** Mengganti *processor* Python kustom dengan *framework* terdedikasi seperti **Apache Flink** atau **KSQLdb** untuk penanganan *stateful processing* (terutama rentang waktu/ *windowing*) yang lebih stabil secara *built-in*.
2. **Skema Sink Connector:** Memanfaatkan **Kafka Connect** (JDBC Sink) untuk menyalurkan hasil ke PostgreSQL secara otomatis dibandingkan membuat lapisan konektor kustom manual.
3. **Cluster Skalabilitas Besar:** Menerapkan replikasi broker menjadi minimal 3 node untuk menguji kapabilitas failover.